

TinyML Smart Pantry & Waste Reducer

An Engineering Journey: Physical Constraints, Optimization, and Lessons Learned

Hardware Target: Arduino Nano 33 BLE Sense + OV7675 Camera

Team: Daniel Tseng, Miles Chin

Core Vision: Privacy-Preserving Food Tracking

The Pantry Problem

- **Goal:** Automatically log groceries at the pantry entrypoint and estimate expiration using USDA FoodKeeper to reduce waste.
- **Edge Privacy:** Perform all computer vision inference locally on-device. Images never leave the microcontroller.
- **Trigger Flow:** Pantry door sensor activates the camera, captures a short image, and transmits a simple event label + timestamp over BLE/Serial.

Target Hardware Constraints

- **Processor:** ARM Cortex-M4F microcontroller running at 64 MHz.
- **Flash Memory:** 1 MB - plenty of space for compiling optimized neural net binaries.
- **SRAM Memory:** 256 KB total. Crucially, Mbed OS and BLE firmware consume ~120-160 KB, leaving only 80-100 KB for model activations!
- **Sensor:** OV7675 image sensor capturing low-resolution QQVGA frames.

Model Design Iterations & Optimization

3-Layer Grayscale

- Input: 96x96x1 (Grayscale)
- Architecture: 3 Conv2D layers with Batch Normalization & ReLU
- Knowledge Distillation: Student model trained to mimic a larger MobileNetV2 RGB teacher
- Quantization: Post-training quantized to 8-bit integer weights/activations

3-Layer RGB

- Input: 96x96x3 (RGB Color)
- Architecture: 3 Conv2D layers with Batch Normalization & ReLU
- Why Color: Added RGB channels to provide color boundaries (essential for telling foods apart)
- Quantization: Quantization-aware training (QAT) to maintain accuracy

4-Layer RGB

- Input: 96x96x3 (RGB Color)
- Architecture: Expanded to 4 Conv2D layers for higher capacity
- Goal: Squeeze maximum capacity to correctly classify complex dishes
- Quantization: QAT to INT8 for local deployment

Model Architecture Comparison



The Results & The Physical SRAM Wall

Model Configuration	Input Space	Val Accuracy (QAT)	Binary Size (Flash)
3-Layer Grayscale (Distilled)	96x96x1	45.6%	~149 KB
3-Layer RGB (Float)	96x96x3	58.0%	~150 KB
4-Layer RGB (Best Capacity)	96x96x3	72.2%	~150 KB

Critical Bottleneck Analysis: RAM vs. Flash

In TinyML, model binary size (stored in Flash) is rarely the primary constraint; activation memory (RAM) is the hard boundary. While our 4-layer RGB model compressed to 150 KB and easily fit within the 1 MB Flash, its peak activation tensors required ~323 KB. This far exceeds the available SRAM on the Arduino, causing compilation and runtime allocation failures.

The Pivot & Dataset Domain Gaps

The Pivot: Pizza vs. Apple Pie

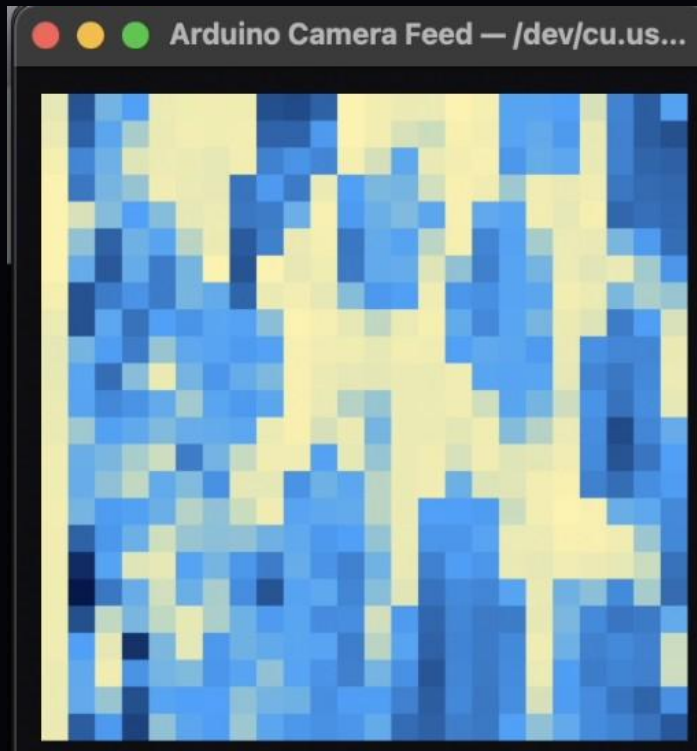
- **Binary Scope:** To bypass memory constraints, we reduced the problem to a simple binary task (Pizza vs. Apple Pie).
- **Feature Overlap:** Both foods are round, crust-based, and baked. Distinguishing them without color (grayscale) and at 96x96 resolution remained highly unreliable.
- **Resolution Limits:** 96x96 resolution completely washes away the fine texture detail required to identify structural features of dishes.

Dataset Domain Mismatch

- **Generalist Data:** Food-101 is web-scraped for general computer vision tasks, not industrial edge sensors.
- **Label Noise:** Many images contain busy backgrounds, macro shots, or people (e.g. a person taking a selfie with a cake is labeled 'cheesecake').
- **Pantry Gap:** A network trained on diverse web photos cannot bridge the domain gap to top-down, fixed-lighting pantry camera captures.

Demo: What the Hardware Actually Sees

A PIZZA (a pizza... right?)



The Reality of 'Hardware Eye'

- **On-Device Print:** Generated directly on the Arduino Nano using our `visualDebugPrint()` helper, which outputs a 24x24 ASCII grid downsampled from the 96x96 grayscale buffer.
- **Contrast & Scale:** Highlights the blocky, low-contrast grayscale crop that the model is forced to interpret.
- **Explainable Failure:** Visualizing this input makes it immediately clear why classification fails: color, edge, and texture details are completely washed away, leaving only generic shapes.

Real Snapshot: Reality vs. Camera View



Visual Gap in Practice

- **Human Perspective:** Full-resolution, high-contrast, color-rich image where food details and boundaries are distinct.
- **Microcontroller View:** Grayscale, 96x96 resolution downsampled image where colors merge and edges blur.
- **Why It Matters:** Differentiating complex dishes like pizza vs. cheesecake requires color and texture. When reduced to a blocky silhouette, the classification boundaries overlap completely, leading to high-uncertainty predictions.

DEMO TIME!

Let's watch the real-time pantry logging in action...

Lessons Learned & Edge CV Realities

Key Takeaways

- **RAM is the Hard Constraint:** In TinyML, memory footprint of weights (Flash) is secondary. Activation tensors (RAM) determine model feasibility.
- **Sufficient Feature Size:** Vision classification requires minimum information capacity. Grayscale 96x96 images lack the entropy required to differentiate complex dishes.
- **Constraint-Driven Scope:** CV on sub-256KB RAM devices is only practical for high-contrast, geometric shapes under rigid lighting (e.g., bar codes, simple gestures).

Future Directions

- **Hardware Upgrades:** Select microcontrollers with external PSRAM (e.g., ESP32-S3) to support larger activation buffers without scaling down resolution.
- **Sensor Fusion:** Integrate low-power non-vision sensors, such as gas/volatile compound sensors, weight cells, or temperature sensors to track freshness directly.
- **Custom In-Situ Data:** Collect a custom in-situ dataset inside the fridge itself rather than relying on noisy web-scraped databases like Food-101.